

# Título del Libro

Subtítulo o descripción

Autor/a

Nombre de la Editorial o Institución

2 de septiembre de 2026



# Índice general

|        |                         |   |
|--------|-------------------------|---|
| 0.0.1. | Librerías . . . . .     | 5 |
| 0.0.2. | Entity . . . . .        | 5 |
| 0.0.3. | Architecture . . . . .  | 6 |
| 0.0.4. | Tipos . . . . .         | 7 |
| 0.1.   | Documentación . . . . . | 8 |



## **Introducción**

El objetivo de este libro es dar una perspectiva más avazada de VHDL.

Este libro está estructurado con los siguientes apartados:

- Estructura de VHDL
- Elementos avanzados
  - Transceivers
- Documentación
- Ejemplos



**VHDL**

**VHDL**

## Estructuras VHDL

Un proyecto en VHDL se estructura de la siguiente forma.

Un proyecto en VHDL se estructura en 3 partes:

1. Importar las librerías.

Todos los proyectos en VHDL incorporan diferentes librerías para poder utilizar los tipos.

2. Definición de puertos y genéricos.

En este apartado se definen dos cosas: los genéricos y los puertos. Los genéricos son constantes utilizadas para escalar los módulos en VHDL.

Los genéricos en muchos casos son utilizados como constantes generales como en otros lenguajes. Esto es un problema que a la larga genera problemas de escalabilidad.

3. Definición de la arquitectura.

Dentro de la arquitectura se define la operatividad del módulo FW

```
library ieee;
use ieee.std_logic_1164.all;

entity modulo is
    generic(

    );
    port(

    );
end entity;

architecture arch_modulo of modulo is

begin

end architecture;
```

### 0.0.1. Librerías

Las librerías en VHDL incorporan los elementos necesarios para poder utilizar el FW.

```
librarie ieee;  
use ieee.std_logic_1164.all;
```

Para incluir librerías propias se incorporan utilizando la librería *work*.

```
use work.package.all;
```

Los fabricantes de FPGAs o de ASICs suelen crear sus propias librerías para acceder a recursos propios, como por ejemplo la librería *UNISIM* de Xilinx(AMD).

### 0.0.2. Entity

En la entidad se compone de dos partes.

#### Generic

```
generic(  
    G_WIDTH  : integer := 8;  
    G_DATA   : integer range 0 to 10 := 8;  
    G_TEXT   : string := "YES"  
);
```

#### Port

En este apartado se definen los puertos del módulo que tienen señales de entrada, salida o bidireccionales.

Entre estos puertos suelen incorporar puertos de reloj, de reset o de habilitación.

Los puertos de reset de normal son activos a nivel bajo ('0').

```
port(  
    PUERTO_A_I  : in std_logic;  
    PUERTO_B_0  : out std_logic_vector(G_WIDTH-1 downto 0);  
    PUERTO_C_IO : inout std_logic
```

);

### **0.0.3. Architecture**

La arquitectura define el comportamiento interno del FW.

**0.0.4. Tipos**

En este apartado

## 0.1. Documentación

Un apartado que no muchas veces se comenta es la forma de documentación.

La forma más utilizada para poder documentar VHDL es utilizando Doxygen.

Doxygen es una herramienta Open Source utilizada en la documentación de un motón de lenguajes como C, C++, VHDL, etc. Para hacer esta documentación se utilizan los comentarios de código.

Para poder documentar con Doxygen es necesario utilizar el formato en los comentarios precedidos por '`!--`'.

`--! Esto es un comentario en formato Doxygen`